# 第1回

## Webアプリの基本と Flask 導入

- 科目: Web アプリケーション脆弱性演習
- テーマ: Web アプリの基本構造を理解する
- 目標: Flask アプリを起動し、画面遷移とコードの対応を説明できる

# 今日の到達目標

- Web アプリが「ブラウザ」と「サーバ」で動くことを説明できる
- HTTP リクエストとレスポンスの流れを説明できる
- Flask アプリを起動できる
- 画面とルーティングの対応をコードから読める
- 今後の演習で使う教材アプリの全体像をつかめる

# 今日扱う内容

1. Web アプリの基本
2. HTTP の基本
3. Flask とは何か
4. GitHub から教材を取得する
5. 画面を触って構造を確認
6. コードの読み方
7. 演習

# Web アプリとは

- ブラウザからサーバへ要求を送る
- サーバが処理して結果を返す
- ブラウザは受け取った HTML を表示する

例:

- `/login` を開く
- ログインフォームを送信する
- サーバが認証して `/me` を返す

# まず押さえたい用語

- クライアント:
  - ブラウザ
- サーバ:
  - Flask アプリ
- リクエスト:
  - ブラウザからの要求
- レスポンス:
  - サーバからの返答
- ルーティング:
  - URL ごとに処理を分ける仕組み

# HTTP の最小イメージ

ブラウザが URL を開く:

```
GET /login HTTP/1.1
```

サーバが画面を返す:

```
HTTP/1.1 200 OK
Content-Type: text/html
```

フォーム送信:

```
POST /login
```

# GET と POST

- `GET`
  - 画面表示や情報取得で使う
  - URL を開く操作に多い
- `POST`
  - 状態変更を伴う処理で使う
  - ログイン、ログアウト、投稿作成など

今日のアプリでも:

- `GET /login`
- `POST /login`
- `POST /logout`

# Flask とは

- Python の軽量 Web フレームワーク
- 少ないコードで Web アプリを作れる
- ルーティングやテンプレートが分かりやすい
- 教材として「何が起きているか」を追いやすい

今回の授業では:

- Flask を使って演習用 Web アプリを読む
- その上で脆弱性演習を重ねる

# 今日使う教材アプリ

今後の授業では、このアプリを段階的に使う。

- ログイン
- ログアウト
- 認証済みページ
- ユーザ検索
- 掲示板
- プロフィール更新
- lab-settings

今日の焦点:

- 起動できること
- 画面とコードが対応すること

# 教材の取得元

教材は GitHub の public repository から取得する。

- Repository:
  - https://github.com/koide55/LectureWebApp001

授業では、この repository を手元に clone して使う。

# 最初に行うこと

```
git clone https://github.com/koide55/LectureWebApp001.git
cd LectureWebApp001
```

このあとで Python 環境を作り、アプリを起動する。

# ディレクトリ構成

```
LectureWebApp001/
  app/
    __init__.py
    routes.py
    templates/
    services/
    db/
  run.py
  requirements.txt
```

重要:

- `run.py` : 起動入口
- `app/__init__.py` : Flask アプリ生成
- `app/routes.py` : 画面ごとの処理
- `app/templates/` : HTML テンプレート

# 起動手順

```
git clone https://github.com/koide55/LectureWebApp001.git
cd LectureWebApp001
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python3 -m app.db.seed
python3 run.py
```

ブラウザで開く:

- http://127.0.0.1:5000

# 初期ユーザ

- `alice / alicepass`
- `bob / bobpass`
- `admin / adminpass`

まずは `alice` でログインしてみる。

# 最初のハンズオン

1. repository を clone する
2. 仮想環境を作る
3. 依存関係を入れる
4. seed を実行する
5. アプリを起動する
6. トップページを開く
7. `alice / alicepass` でログインする
8. `My Page` と `Lab Settings` を開く

観察ポイント:

- どのコマンドで何をしているか
- URL が変わる
- 画面ごとに役割が違う
- ログイン前後で見え方が変わる

# コードの入口 1

## `run.py`

```python
from app import create_app

app = create_app()

if __name__ == "__main__":
    app.run(debug=True)
```

ポイント:

- `create_app()` で Flask アプリを作る
- `app.run()` で開発用サーバを起動する

# コードの入口 2

## `app/__init__.py`

```python
def create_app(config_class=None):
    from flask import Flask
    from .config import Config
    from .routes import main_bp

    if config_class is None:
        config_class = Config

    app = Flask(__name__, instance_relative_config=True)
    app.config.from_object(config_class)
    app.register_blueprint(main_bp)
    return app
```

ポイント:

- Flask アプリ本体を作る
- 設定を読み込む

# コードの入口 3

## `app/routes.py`

```
main_bp = Blueprint("main", __name__)

@main_bp.get("/")
def index():
    return render_template("index.html")
```

ポイント:

- `"/"` にアクセスしたら `index()` が動く
- `index.html` を返している
- URL と関数が対応している

# ルーティングの読み方

例:

```
@main_bp.route("/login", methods=["GET", "POST"])
def login():
    ...
```

意味:

- `/login` へのアクセスを処理する
- `GET` なら画面表示
- `POST` ならフォーム送信処理

# テンプレートの役割

`render_template("login.html")` は、
`app/templates/login.html` を使って HTML を返す。

つまり:

- Python:
  - 処理の流れを書く
- HTML テンプレート:
  - 画面の見た目を書く

# コード解説

## `login.html`

```
<form action="{{ url_for('main.login') }}" method="post" class="stack">
  <label for="username">Username</label>
  <input id="username" name="username" type="text" required>

  <label for="password">Password</label>
  <input id="password" name="password" type="password" required>

  <button type="submit">Login</button>
</form>
```

ポイント:

- `form` が送信フォーム
- `method="post"` で POST 送信
- `name="username"` などの名前でサーバに値が渡る

# フォーム送信とサーバ処理

```
user, error, cookie_value = attempt_login(
    request.form.get("username", "").strip(),
    request.form.get("password", ""),
)
```

ポイント:

- `request.form` から入力値を取り出す
- サーバ側で認証を行う
- 「入力しただけでログイン」にはならない

# 画面とコードの対応

今日の課題は、次の対応を説明できるようになること。

- トップページ:
  - `index.html`
  - `index()`
- ログインページ:
  - `login.html`
  - `login()`
- マイページ:
  - `me.html`
  - `me()`

# このアプリで今後学ぶこと

- 認証
- セッション
- Cookie
- SQLインジェクション
- CSRF
- 反射型 XSS
- 蓄積型 XSS
- コマンドインジェクション
- 認可不備

今日の段階では:

- まず普通の Web アプリとして読めることが重要

# 演習 1

## repository を取得して起動する

次の手順を実行する。

```
git clone https://github.com/koide55/LectureWebApp001.git
cd LectureWebApp001
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python3 -m app.db.seed
python3 run.py
```

できたらブラウザでトップページを開く。

# 演習 2

## 画面とURLの対応を確認する

次の表を埋める。

| 画面 | URL | 主な役割 |
|---|---|---|
| トップページ | | |
| ログイン | | |
| マイページ | | |
| プロフィール | | |
| 掲示板 | | |
| Lab Settings | | |

# 演習 3

## ルーティングを探す

`app/routes.py` を開いて、次を探す。

1. `/`
2. `/login`
3. `/me`
4. `/profile`
5. `/lab-settings`

確認すること:

- どの関数が担当しているか
- `GET` か `POST` か
- どのテンプレートを返しているか

# 演習 4

## テンプレートを読む

次のテンプレートを開いて、共通点と違いを確認する。

- `app/templates/base.html`
- `app/templates/login.html`
- `app/templates/me.html`

見るポイント:

- ナビゲーション
- フォーム
- 表示しているデータ

# 演習 5

## コードを追う

以下の問いに答える。

1. アプリはどこで生成されているか
2. `/login` の表示処理はどこか
3. `/login` の送信処理はどこか
4. `/me` はどのテンプレートを使うか

# 演習 6

## 動作確認

ブラウザで次を試す。

1. ログイン前に `/me` を開く
2. ログイン後に `/me` を開く
3. `Profile` を開く
4. `Lab Settings` を開く

考えること:

- ログイン前後で何が違うか
- なぜ違いがあるのか

# 今日のコード解説まとめ

- `run.py`
  - 起動入口
- `app/__init__.py`
  - アプリ生成
- `app/routes.py`
  - URL と処理の対応
- `app/templates/*.html`
  - 画面の見た目

つまり:

- Web アプリは「URL」「処理」「画面」の対応で読む

# 次回予告

- ログインとログアウトの仕組み
- 認証と認可の違い
- 保護されたページへのアクセス制御

# 宿題

1. `app/routes.py` を読んで、少なくとも 5 つの URL と関数の対応をメモする
2. `app/templates/base.html` を読み、全ページ共通の要素を列挙する
3. なぜ `/login` は `GET` と `POST` の両方を使うのか考える

# 教員メモ

- 1回目は脆弱性の話に深く入りすぎない
- 「まず普通の Web アプリとして理解する」を重視する
- 余裕があればブラウザの開発者ツールでフォーム送信を見せる
- 次回の認証回につなぐため、`/login` と `/me` の流れを丁寧に確認する